AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS

Amrita School of Computing

Coding Practice

Activity Sheets

(For the Batch 2023-2027)

Name:

Roll No.:

Department:

Activity Sheet 05 **Start Date: 24/05/2026 End Date: 27/05/2026.**

Topic: Dynamic Programming [Easy + Medium Coding Questions]

Instructions

The Mock Test syllabus for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test [Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the completed Activity sheets before the deadline in the MS-Form link shared by your class Advisor.

Activity Sheet: 05 - Coding Questions from Dynamic Programming [Easy + Medium Questions]

Each Activity Sheets contains total of 15 Questions.

Platform: Any Online Compiler of your choice – GDB Compiler

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take screen shot of the code and the output and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in PDF Format and the file name should be your roll number.

Course Instructors: CARE TEAM
Dr.R.Annamalai Assistant Professor (Sl.Grade) |CSE-AIE
Dr.J.Umamageswaran Assistant Professor (Sr.Grade) | CSE-CT
Ms.D. Sasikala Assistant Professor | CSE-AIE
Mr.NirmalRaj Assistant Professor | CSE-CYS

1. Given an integer array nums, return the length of the longest strictly increasing subsequence.

   Example 1:

   Input: nums = [10,9,2,5,3,7,101,18]
   Output: 4
   Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.
   Example 2:
   Input: nums = [0,1,0,3,2,3]
   Output: 4
   Example 3:
   Input: nums = [7,7,7,7,7,7,7]
   Output: 1

Code:

Output

2. You are given an integer array coin representing coins of different denominations and an integer amount representing a total amount of money.

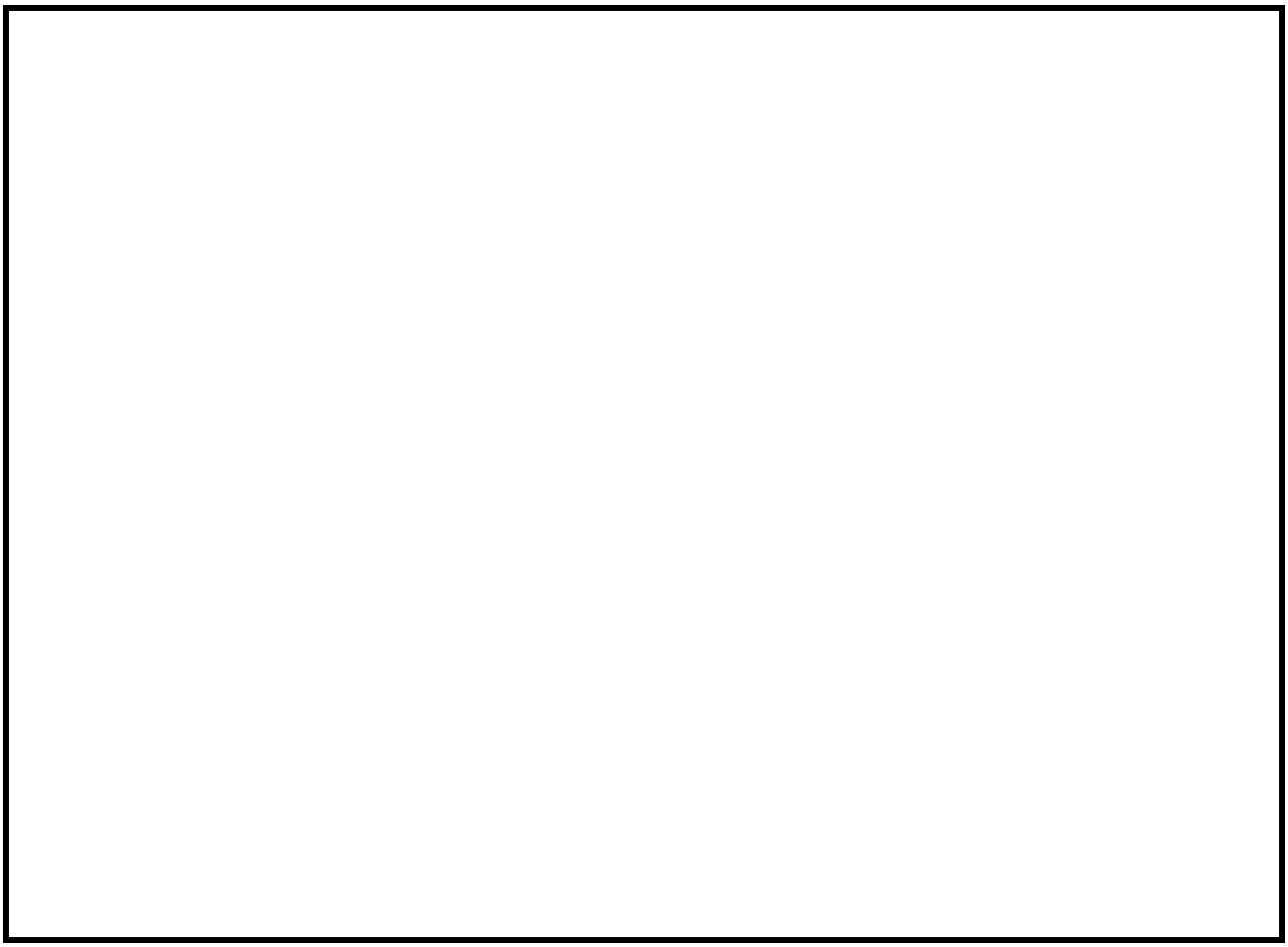
   Return the fewest number of coins that you need to make up that amount. If that amount of money Cannot be made by any combination of the coins, return -1.
   You may assume that you have a infinite number of each kind of coin.
   Example 1:
   Input:
   Coins=[1,2,5]
   Amount=11
   Output: 3
   Explanation: 11=5+5+1

Example 2:
Input :
Coins=[2]
Amount =3
Output: 1

Code:

Output

3.



3. Given two strings ‘s1‘ and ‘s2‘, find the length of the longest common substring.

Example:

Input: s1 = “GeeksforGeeks”, s2 = “GeeksQuiz”
Output : 5
Explanation:
The longest common substring is “Geeks” and is of length 5.

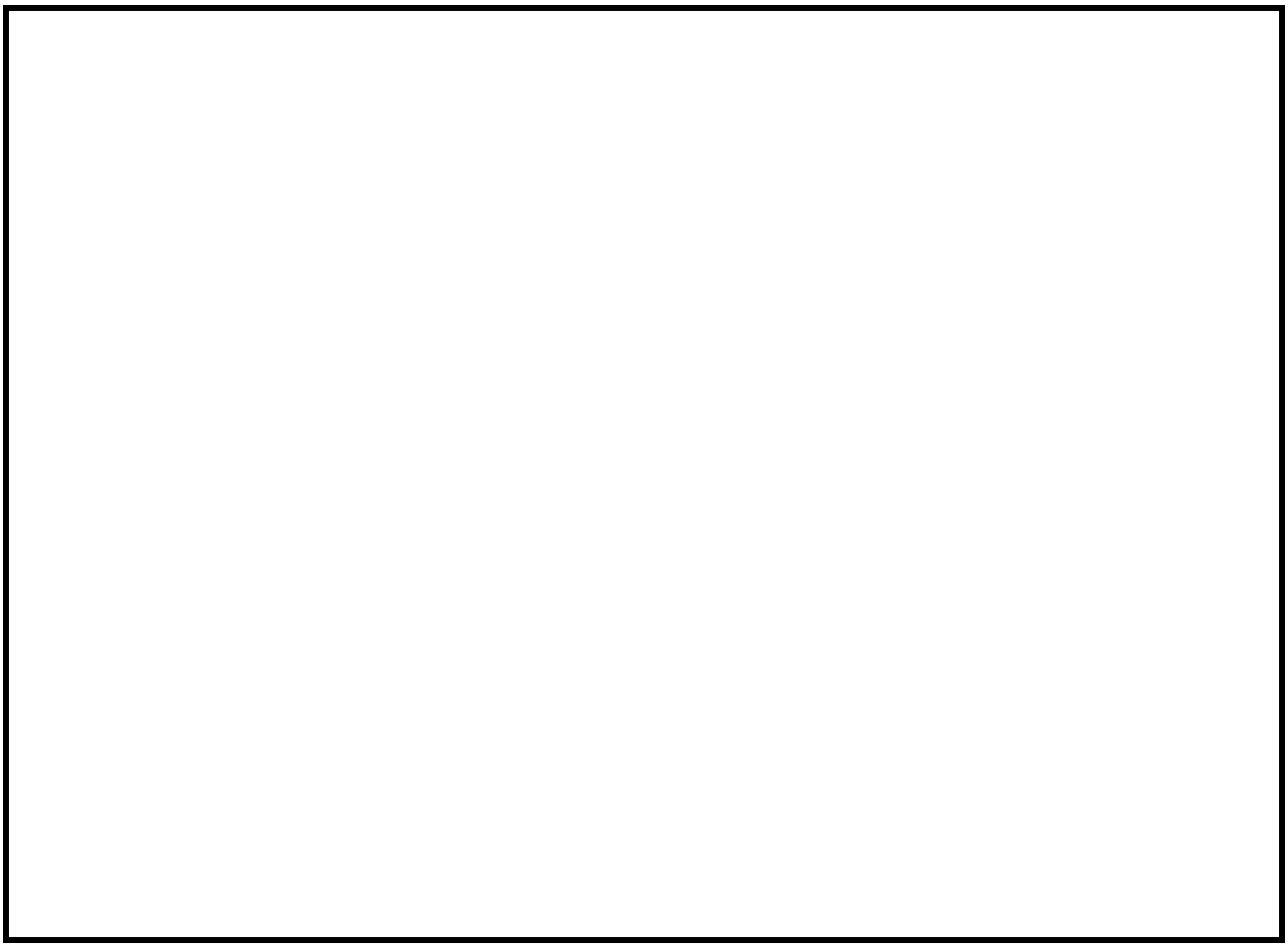
Input: s1 = “abcdxyz”, s2 = “xyzabcd”
Output : 4
Explanation:
The longest common substring is “abcd” and is of length 4.

Input: s1 = “abc”, s2 = “”
Output : 0.

Code:

Output

1.



4. Given an array arr[] where each element shows the amount of money in a row of houses built in a line. A robber wants to steal money, but cannot rob two houses adjacent to each other because it will set of an alarm.
Find the maximum money the robber can steal without robbing two adjacent houses.

Examples:

Input: arr[] = [6, 7, 1, 3, 8, 2, 4]
Output: 19
Explanation: The thief will steal from house 1, 3, 5 and 7, total money = 6 + 1 + 8 + 4 = 19.

Input: arr[] = [5, 3, 4, 11, 2]
Output: 16
Explanation: Thief will steal from house 1 and 4, total money = 5 + 11 = 16.

Code:

Output

5. Given a rod of length n and an array price[]. price[i] denotes the price of a piece of length i. Determine the maximum amount obtained by cutting the rod into pieces and selling the pieces.
   Note: price[0] is always 0.
   Input: price[] = [0, 1, 5, 8, 9, 10, 17, 17, 20]
   Output: 22
   Explanation: The maximum obtainable value is 22 by cutting in two pieces of lengths 2 and 6, i.e., 5 + 17 = 22.
   Input : price[] = [0, 3, 5, 8, 9, 10, 17, 17, 20]
   Output : 24
   *Explanation : The maximum obtainable value is 24 by cutting the rod into 8 pieces of length 1, i.e, 8*price[1]= 8*3 = 24.*
   Input : price[] = [0, 3]
   Output : 3
   Explanation: There is only 1 way to pick a piece of length 1.

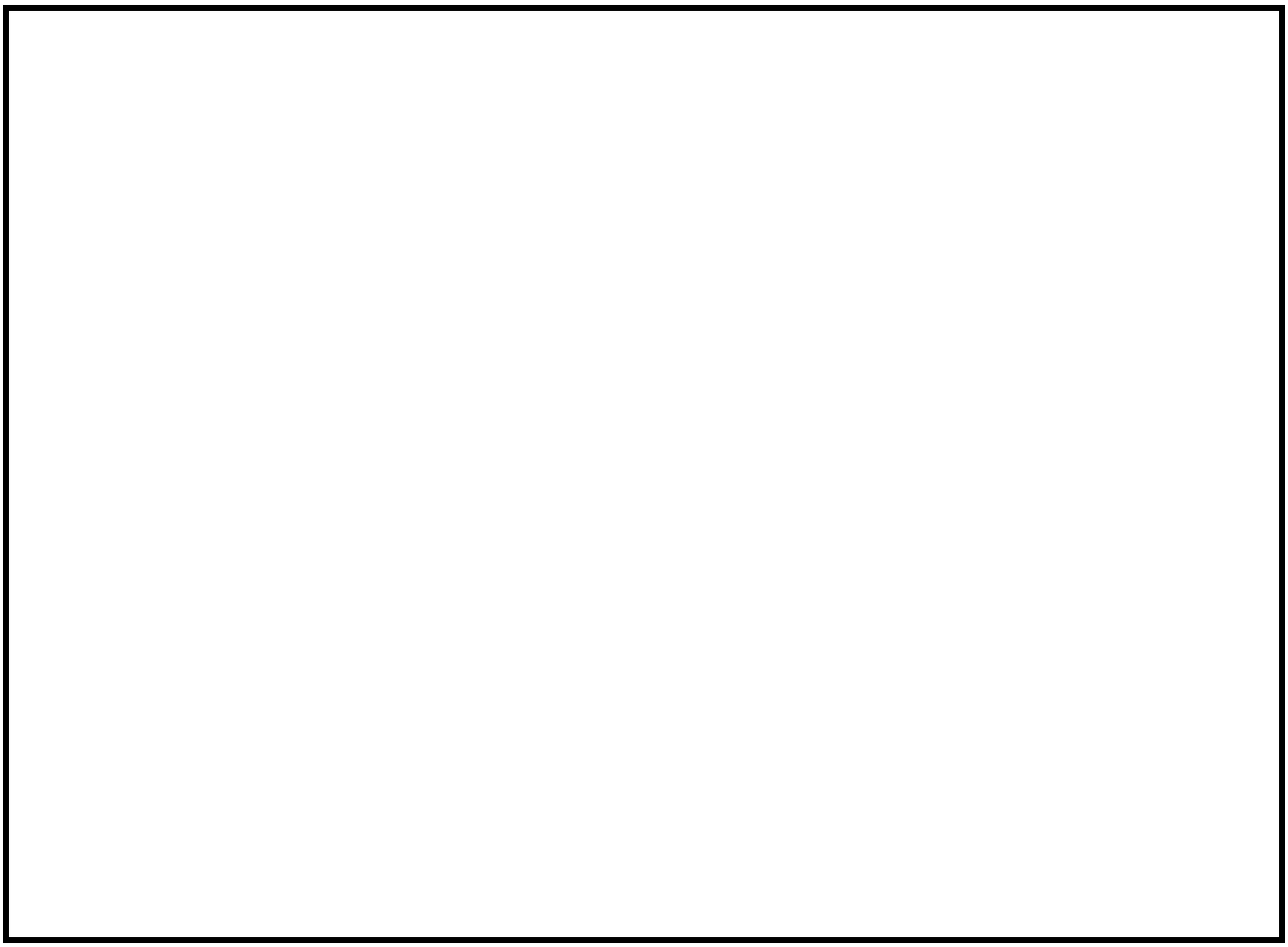
Code:

Output:

6. Given **n** stairs, and a person standing at the ground wants to climb stairs to reach the top. The person can climb either **1 stair** or **2 stairs** at a time, count the number of ways the person can reach at the top.
   Examples:
   Input: *n = 1*
   Output: *1*
   Explanation*: There is only one way to climb 1 stair.*
   Input: *n = 2*
   Output: *2*
   Explanation*: There are two ways to reach 2th stair: {1, 1} and {2}.*
   Input: *n = 4*
   Output: *5*
   Explanation*: There are five ways to reach 4th stair: {1, 1, 1, 1}, {1, 1, 2}, {2, 1, 1}, {1, 2, 1} and {2, 2}.*

Code:

Output:



7. Given an array **arr[]** of non-negative integers and a value **sum**, the task is to check if there is a **subset** of the given array whose sum is equal to the given **sum**.
   Examples:

Input: *arr[] = [3, 34, 4, 12, 5, 2], sum = 9*
Output: *True*
Explanation: *There is a subset (4, 5) with sum 9.*
Input: *arr[] = [3, 34, 4, 12, 5, 2], sum = 30*
Output: *False*
Explanation: *There is no subset that add up to 30.*

Code:

Output:

8. *Let **1 maps to 'A', 2 maps to 'B', ..., 26 to 'Z'.** Given a **digit sequence**, **count** the number of possible **decodings** of the given digit sequence.*
   *Consider the input string **"123"**. There are three valid ways to decode it:*
   "ABC"**: The grouping is (1, 2, 3) → **'A', 'B', 'C'
   "AW"**: The grouping is (1, 23) → **'A', 'W'
   "LC"**: The grouping is (12, 3) → **'L', 'C'
   ***Note:** Groupings that contain **invalid codes** (e.g., "0" by itself or numbers greater than "26") are not allowed.*
   *For instance, the string **"230"** is **invalid** because **"0"** cannot stand alone, and **"30" is greater than "26",** so it cannot represent any letter. The task is to find the **total number of valid ways** to decode a given string.*
   Example:
   ***Input:** digits = "121"*
   ***Output:** 3*
   ***Explanation:** The possible decodings are "ABA", "AU", "LA"*
   ***Input:** digits = "1234"*
   ***Output:** 3*
   ***Explanation:** The possible decodings are "ABCD", "LCD", "AWD"*

Code:

Output:

9. *Given an array **arr[]** of non-negative integers, where each element represents the maximum number of steps you can jump forward from that index, determine the minimum number of jumps required to reach the **last index** starting from the **first index**. If it is not possible to reach the end, return **-1**.*

Examples:

Input: *arr[] = [1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]*

Output: *3*

Explanation: *First jump from 1st element to 2nd element with value 3. From here we jump to 5th element with value 9, and from here we will jump to the last.*

Input: *arr []= [1, 4, 3, 2, 6, 7]*

Output: *2*

Explanation: *First we jump from the 1st to 2nd element and then jump to the last element.*

Input: *arr[] = [0, 10, 20]*

Output: *-1*

Explanation: *We cannot go anywhere from the 1st element.*

Code:

Output:

10. *Given a grid of size **m x n**, determine the number of distinct paths from the **top-left** corner (0,0) to the **bottom-right** corner (m-1, n-1). At each step, one can either move down or right.*
    ***Input:** m = 3, n = 3*
    ***Output:** 6*
    ***Explanation:** Let the given input 3*3 grid is filled as such:*
    A B C
    D E F
    G H I
    The possible unique paths which exists to reach 'I' from 'A' following above conditions are as follows: ABCFI, ABEHI, ADGHI, ADEFI, ADEHI, ABEFI.
    ***Input:** m = 2, n = 3*
    ***Output:** 3*
    ***Explanation:** Let the given input 2*3 grid is filled as such:*
    A B C
    D E F
    The possible unique paths which exists to reach 'F' from 'A' following above conditions are as follows: ABCF, ADEF and ABEF.
    ***Input:** m = 1, n = 4*

Output: *1*
Explanation: *Let the given input 1*4 grid is filled as such:*
A B C D
The only possible unique path is ABCD.

Code:

Output:

11. *Given an array **arr[]** of size **n**, find the **length** of the Longest Increasing Subsequence (LIS) i.e., the longest possible subsequence in which the elements of the subsequence are sorted in **strictly increasing order**.*

Examples:
Input: *arr[] = [3, 10, 2, 1, 20]*
Output: *3*
Explanation: *The longest increasing subsequence is 3, 10, 20*

Input: *arr[] = [30, 20, 10]*
Output:*1*
Explanation: *The longest increasing subsequences are [30], [20] and [10]*
Input: *arr[] = [2, 2, 2]*
Output: *1*
Explanation: *We consider only strictly increasing subsequences, therefore the longest increasing*

subsequence is [2].

Input: *arr[] = [3, 4, 5, 1, 2, 3, 4]*
Output: *4*
Explanation: *The longest strictly increasing subsequence is [1, 2, 3, 4], which gives a maximum length of 4. (Note: [3, 4, 5] is also an increasing subsequence, but its length is only 3).*

Code:

Output:

*12. Given two arrays, **a[] and b[]**, find the length of the **longest common increasing subsequence(LCIS)**. A **subsequence** is a sequence that appears in the same relative order in both arrays, but not necessarily consecutively.*
The Longest Common Increasing Subsequence (LCIS) is defined as the longest sequence that appears in both arrays and whose elements are in strictly increasing order.
Examples:
***Input:** a[] = [3, 4, 9, 1], b[] = [5, 3, 8, 9, 10, 2, 1]*
***Output:** 2*
***Explanation:** The longest increasing subsequence that is common is {3, 9} and its length is 2.*
***Input:** a[] = [1, 1, 4, 3], b[] = [1, 1, 3, 4]*
***Output:** 2*
***Explanation:** There are two common subsequences {1, 4} and {1, 3} both of length 2.*

Code:

Output:

12. Given a string **s** and y a dictionary of **n** words **dictionary**, check if s can be segmented into a sequence of valid words from the dictionary, separated by spaces.
Examples:
Input: *s = "ilike", dictionary[] = ["i", "like", "gfg"]*
Output: *true*
Explanation: *The string can be segmented as "i like".*
Input*: s = "ilikegfg", dictionary[] = ["i", "like", "man", "india", "gfg"]*
Output*: true*
Explanation: *The string can be segmented as "i like gfg".*
Input: *"ilikemangoes", dictionary = ["i", "like", "gfg"]*
Output: *false*
Explanation: *The string cannot be segmented.*

Code:

Output:

14. Given an array **arr[],** check if it can be **partitioned** into two parts such that the **sum** of elements in both parts is the same.
Note: Each element is present in either the first subset or the second subset, but not in both.
Examples:
Input: *arr[] = [1, 5, 11, 5]*
Output: *true*
Explanation: *The array can be partitioned as [1, 5, 5] and [11] and the sum of both the subsets are equal.*
Input: *arr[] = [1, 5, 3]*
Output: *false*
Explanation: *The array cannot be partitioned into equal sum sets.*

Code:

Output:

15. Given a string **s**, the task is to find the **minimum** number of characters to be inserted to convert it to a Palindrome.

Examples:
Input: *s = "geeks"*
Output: *3*
Explanation: *"skgeegks" is a palindromic string, which requires 3 insertions.*
Input: *s= "abcd"*
Output: *3*
Explanation: *"abcdcba" is a palindromic string.*
Input: *s= "aba"*
Output: *0*
Explanation: *Given string is already a palindrome hence no insertions are required.*

Code:

Output: